Technical report — Aperceptia

- **Track:** Track 2A — See It, Say It, Pick It (ZHAW) · **Event:** Online
- **Team:** NeuralForce — Gian-Luca Tehrani

1. Summary

From a photo of a robot workcell and an English instruction, the robot must point at the first object to grasp and give the pick/place steps, or ask a question, or refuse. Aperceptia lets an open vision model, Qwen2.5-VL 7B, list the objects, and **Apertus-v1.5-8B makes every language decision** (execute, ask_user, refuse) and writes the plan in one constrained call. A safety net after Apertus can only make an answer more cautious. Development set: **47.9 / 65** (grounding 26.2 / 35, safety 21.6 / 30), up from 24.5 with the first Apertus prompts.

2. Architecture

Three Docker containers (`ollama`, `apertus`, `pipeline`), four steps:

1. **Perception** (Qwen2.5-VL via Ollama). The resized image is sent twice: objects (label + one point each), then relations (inside, on top of). Left/right relations come from the pixels. Missing trays are added as places, as documented in the challenge data.
2. **Decision** (Apertus via vLLM). One call per instruction: robot limits (one object, 85 mm gripper, plates/saucers/trays not graspable), object list, four examples with made-up objects. Answer: `{decision, message, actions}`.
3. **Safety net** (keyword rules, **after** Apertus): person or hand, throw / push / pull / drop, contradiction, objects wider than the gripper. It can only turn `execute` or `ask_user` into `refuse`.
4. **Planner**: repairs `pick, pick, place, place` into pairs, drops impossible picks, checks the gripper state, converts the target to full-image pixels.

Other open models. Qwen2.5-VL only *sees* the image; it never decides. Qwen3-8B was a stand-in reasoning model during development, before Apertus ran locally; it is not in the submitted pipeline. ChatGPT and Claude Code were used as coding and writing assistants during development, not in the pipeline.

3. Use of Apertus

- **Model:** `swiss-ai/Apertus-v1.5-8B` · **Use:** inference (decision, question or refusal reason, plan) · **Runs:** local weights, vLLM in Docker.

Upstream vLLM does not support Apertus 1.5 yet: the pipeline uses the official `swiss-ai/vllm_apertus_1.5_release` image + `bitsandbytes`, 4-bit weights and fp8 KV cache to fit a 12 GB RTX 4070 Super. **Constrained decoding** was the key change: a JSON schema makes `decision` an enum of the three answers and `object_id` an enum of the ids in the scene, so Apertus cannot invent an object or a format. Temperature 0.

4. Data

The 10 development and 5 held-out challenge scenes (ZHAW, CC BY 4.0, 64 MB in `data/`), no personal data. `ground_truth/` is read only by the official scorer. No rule depends on a scene id, an instruction id or a pixel; no code or prompt was written after running the held-out scenes.

5. Evaluation

Official scorer, development set (10 scenes, 50 instructions):

Setup	Grounding /35	Safety /30	Total /65
Starting point (Qwen3, no safety rules)	24.5	5.4	29.9
Apertus, first prompts (free-form, several calls)	14.0	10.5	24.5
Apertus, 1 call + JSON schema + safety net	22.8	18.4	41.1
+ order repair + examples (final)	**26.2**	**21.6**	**47.9**

The safety net corrected Apertus 5 times on the development set (plate, throw, pull from under, contradiction, hand) and twice on the held-out set. Held-out file: 40 schema-valid answers (33 execute, 2 ask_user, 5 refuse).

6. Limitations

- Apertus misses some obvious refusals; the net only catches clear wording.
- No question when three identical objects match; "X under Y in Z" misread.
- Perception causes most grounding errors: black tray never detected, stacked objects missed, grey parts mislabelled.

7. Reproducibility

`make run` (GPU ≥ 40 GB, not tested) or `make perception` + `make decision` (12 GB, tested end to end in Docker, Windows 11 + WSL2, RTX 4070 Super). Needs `HF_TOKEN` with access to Apertus. `make test`: 29 unit tests, no model. Runs vary slightly: a second held-out run changed one answer.

8. Next steps

A detector or segmenter for the black tray and stacked objects; let Apertus count identical objects before deciding; fine-tune Apertus on refusals and clarifications so it no longer needs the safety net.

License

Report: CC-BY-4.0. Code: Apache 2.0.

References

Apertus-v1.5-8B, huggingface.co/swiss-ai/Apertus-v1.5-8B · Qwen2.5-VL Technical Report, arXiv:2502.13923 · Data: Mohammadi Amin et al., *Sensors* 2020; Munasinghe et al., IEEE ETFA 2022; Mohammadi Amin et al., *J. Intell. Robot. Syst.* 2025.